

Hardware thread-context switching

L. Sawalha, M. P. Tull and R. D. Barnes

Numerous proposals have advanced fine-grained thread migration as a mechanism to address power, performance, reliability and memory coherence problems. However, exploiting conventional context switch mechanisms carries significant overhead, limiting the granularity of thread movement. Proposed is a novel hardware context switching circuit that enables low-overhead hardware thread migration between cores in a single-chip multiprocessor. This switching circuit supports multiple simultaneous thread switches and can store the context of both currently running and time-multiplexed threads. The circuit drastically reduces the direct cost of context switches.

Introduction: In computer systems, context switching, the task of saving the state of one process or thread and loading the state of another, is time consuming (30–60 μ s by current estimates [1]). This overhead increases the granularity of the time-slices and each process must be appropriated to avoid excessive time during which the system does no useful work. The penalty of context switching includes both a direct cost of saving and loading process state and an indirect cost, which includes the time that is needed to repopulate microarchitectural structures such as caches and predictors. This Letter presents a hardware context switching circuit that includes a shared context unit (SCU). This circuit both accelerates migration of threads between cores in a multi-core processor and reduces the direct cost of context switching within each processing core.

Heterogeneous (or asymmetric) multicore processors (HMPs), processors with more than one type of processing core, can particularly benefit from faster process migration. HMPs offer advantages over homogeneous multicore processors in terms of both power and performance [2]. Differences between cores enable the exploitation of fine-granularity changes in programme behaviour but this requires frequent process migration. In addition to fine-grained scheduling for HMPs [3], faster thread migration benefits many other migration-based techniques including load balancing [4], thermal and power management [5], coherence protocols [6] and manufacturing-fault tolerance [7].

In a multicore processor, rather than relying on operating system (OS) software to perform context switches by assigning or reassigning threads among the processing cores, our proposed hardware circuit supports switching between architectural copies of all cores' registers in the SCU. When a process is migrated from one core to another, the new core will be redirected to the context of that process before migration (each type of core is assumed to have the same quantity and width of CPU registers). The SCU component is shared among a set of cores (n) on a chip multiprocessor and is composed of a number of context sets (CSs) (m). The number of running threads can exceed the number of CSs; therefore, the OS is responsible for switching among additional threads' contexts through the traditional mechanism of copying the context of these threads to memory as they are forced out of the SCU and copying them back to the SCU when they are scheduled for execution.

To accelerate switches between time-multiplexed threads, the number of CSs in the SCU can exceed the number of cores. Other architectural approaches similarly utilise multiple register files including simultaneous multithreading (SMT) and checkpointing. In SMT [8], multiple register files support multiple threads executing simultaneously within the same core. In out-of-order execution with checkpointing, additional copies of register values are used to save the processor state at appropriate points of execution [9]. These copies are used to repair register contents to a previous state when exceptions or branch mispredictions occur.

Each CS in the SCU contains a copy of the processor context including the basic execution environment (584 bytes for an x86 64-bit processor), debug, control and design-specific registers etc. (~1000 bytes). Using CACTI [10], a CS's area was estimated as 1971 nm² for a 32 nm process with each 8-byte access consuming 1.4 pJ. Ideally, the SCU is physically laid out near each of the cores. Because the architectural CPU registers must be closely integrated within the retirement stage of the CPU's pipeline, a long wire delay would be intolerable. For a processor with large numbers of cores, it may be infeasible to share the same context unit for all the cores. Thus, the processor can consist of clusters of cores, such that a few cores form a cluster and share one context unit. To migrate a process/thread from one cluster

to another, the scheduler must copy the entire context of the migrating process/thread from one cluster's context unit to another, possibly requiring OS coordination.

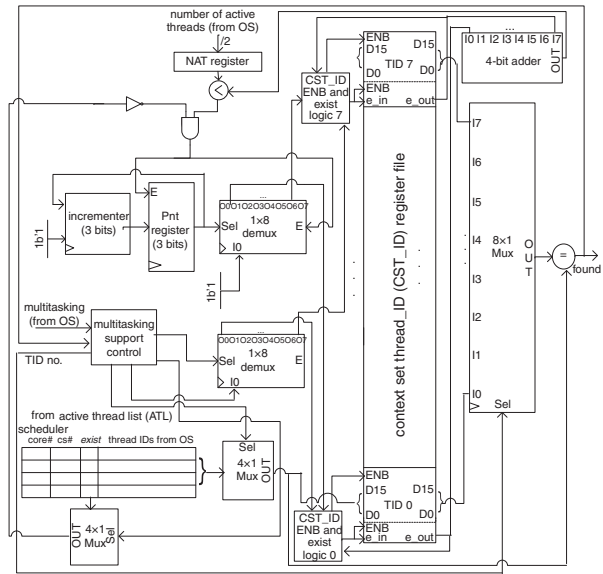


Fig. 1 Components for multitasking support ($n = 4$, $m = 8$)

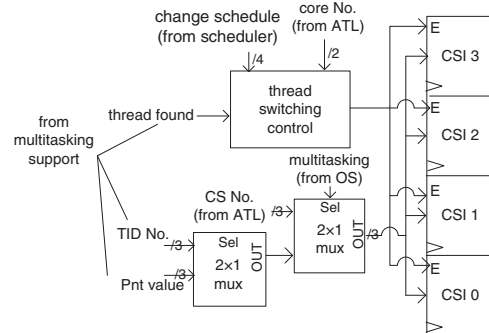


Fig. 2 Components for thread switching ($n = 4$, $m = 8$)

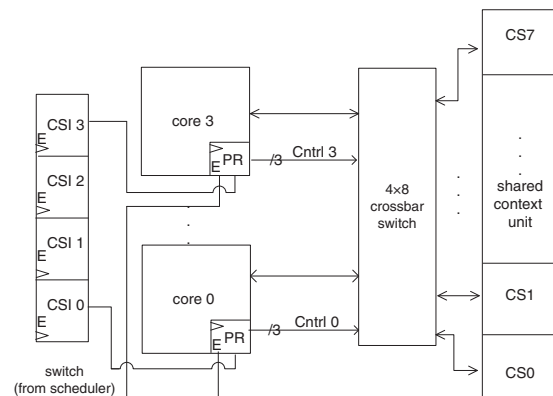


Fig. 3 Context set pointer switching components ($n = 4$, $m = 8$)

Context switching circuit: The thread switching circuit is responsible for switching threads among different cores and consists of control logic, a crossbar switch, additional registers and the SCU. The control logic receives information about threads and their scheduling assignment to different cores from the OS and the scheduler. The scheduler could be part of the OS in software, or could be implemented in hardware or a hypervisor. The additional registers are used to record the scheduling information and the indices to the CSs in the SCU. The thread switching circuit is composed of three main partially overlapping sets of components utilised for context switching, multitasking support, thread migration and CS pointer switch, shown in Fig. 1–3 respectively.

The circuits in the Figures show four cores ($n = 4$) and eight CSs ($m = 8$) but can be extended for reasonable m and n , where $n \leq m$.

In the first set of components, multitasking support, shown in Fig. 1, the OS assigns the IDs of threads that are scheduled to run during the next time interval to an active-thread list (ATL). The number of entries in the ATL is the same as the number of processing cores (n). Along with the thread ID for each entry, a $\lceil \log_2 m \rceil$ -bit context set index, a $\lceil \log_2 n \rceil$ -bit core index and an exist bit are present for each active thread. The context set index is written after each thread is assigned a CS. The scheduler inserts the core number that each active thread is assigned. The exist bit is reset before multitasking processing is started and set whenever the thread, associated with the same entry in the ATL, is found in one of the CSs or that thread is assigned a new CS in the SCU. A *context_set_thread_ID* (CST_ID) register file contains a number of registers equal to the number of CSs (m) in the SCU. Each register is composed of a 16-bit thread ID and a 1-bit exist flag (e). Each register corresponds to one CS in the SCU containing the context (state) of that thread. The CST_ID register file is sequentially searched for a thread ID match from the thread IDs in the ATL. A $\lceil \log_2 n \rceil$ -bit counter (within the multitasking support control in Fig. 1) and an $n \times 1$ multiplexer is used to select one thread ID in the active list every 1 to m cycles (depending on the time required to search for that ID as subsequently explained). Another $\lceil \log_2 m \rceil$ -bit counter is used to point to the current CST_ID register as the CST_ID register file is searched for a match with the thread_ID from the ATL. This counter is reset once IDs match. Each thread ID in the ATL is compared to each thread ID in the register file (one per cycle) until the thread ID is found or all registers are searched. If the thread ID is found, the exist bit is set to one in both the CST_ID register and the corresponding entry in the ATL, and the $\lceil \log_2 n \rceil$ -bit counter is enabled. The next thread_ID is then searched. The process of searching all thread IDs takes between $(1 + 2 + \dots + l)$ and $m \times l$ cycles, where l is the number of active threads ($l \leq n$). After all thread IDs are searched, the number of the exist bits in the register file that are set is calculated using an $\lceil m/2 \rceil$ -bit full adder. The $\lceil \log_2 n \rceil$ least significant bits of the summation are compared to the value of the number of active threads register (NAT). If the result is less than the NAT value, this means that some threads do not exist in the SCU. Each thread that does not exist in the SCU is then assigned an available CS in the SCU and a corresponding spot in the CST_ID register file. The CSs in the SCU are assigned round-robin to ensure fair utilisation of all sets.

Thread migration in Fig. 2 is divided into two scenarios, migration due to multitasking and migration due to dynamic thread reassignment to adapt to the dynamic changes in workload behaviour (e.g. to increase performance or reduce power consumption in an HMP [11]). In the case of multitasking, after all threads are assigned a CS from the SCU pool, the scheduler stores the new assignment of active threads to the processing cores in the ATL. Thread IDs in the CST_ID register file are compared to thread IDs in the ATL. The indices to the CSs of active threads are then recorded in n context set index (CSI) registers shown in Fig. 2, each of $\lceil \log_2 m \rceil$ bits size. Each CSI register corresponds to one of the n cores. In the second scenario, if the scheduler changes the assignment of the same threads that are currently executing, it writes the new core numbers for each thread ID in the ATL and asserts changeSchedule signals for each core that is switching to another thread. Next, the CSs' indices in the CSI registers are updated using the information recorded in the ATL in parallel.

After all pipelines that are switching to different threads are either flushed or drained, a switch signal is set. Each core receives a new index to the proper CS for a new thread to execute. The index to the CS is stored in a $\lceil \log_2 m \rceil$ bit pointer register (PR) as shown in Fig. 3.

Each PR register is updated with a new value from its corresponding register in the CSI register file after the switch signal is set. Next, all cores point to their proper CSs for the applications/threads they are executing through an $n \times m$ crossbar switch. The crossbar switch is capable of connecting all n cores to any n CSs out of the m sets in the context unit simultaneously. Each core points to only one CS at a time but is capable of accessing any CS through the crossbar switch. Because a cluster of cores is assumed to contain only a few cores sharing a single SCU, the cost of the crossbar switch is limited.

Conclusion: A novel thread context switching circuit is proposed. The circuit supports multitasking by containing more CSs than the number of cores, supports migrating all threads that are executing on processor cores simultaneously, and supports fine-granularity thread scheduling by rapidly migrating the contexts of all executing threads simultaneously. Only two cycles are required to switch the executing threads among cores (in addition to the time required to flush processor pipelines). Additionally, the maximum number of cycles required for thread assignment due to multitasking is $m \times l + 2m - 1$, where l is the active number of threads and m is the number of CSs. Our hardware design results in a large reduction in the latency of context switching among multiple cores, compared to tens of thousands of cycles for the direct cost of context switching by the OS.

© The Institution of Engineering and Technology 2013

1 October 2012

doi: 10.1049/el.2012.2887

One or more of the Figures in this Letter are available in colour online.

L. Sawalha, M. P. Tull and R. D. Barnes (*The University of Oklahoma, Norman, OK 73019, USA*)

E-mail: ron@ou.edu

References

- 1 Ronen, R.: 'The practical challenges of heterogeneous architectures,' Invited Talk at SHAW-3, New Orleans, LA, USA, February 2012
- 2 Kumar, R. *et al.*: 'Heterogeneous chip multiprocessors,' *Computer*, 2005, **38**, 11, pp. 32–38
- 3 Sawalha, L. *et al.*: 'Thread scheduling for heterogeneous multicore processors using phase identification,' *SIGMETRICS PER*, 2011, **39**, 3, pp. 125–127
- 4 Sanchez, D. *et al.*: 'Flexible architectural support for fine-grain scheduling,' *AspLos*, Pittsburg, PA, Pennsylvania, pp. 311–322, March 2010
- 5 Coşkun, A.K. *et al.*: 'Static and dynamic temperature-aware scheduling for multiprocessor SoCs,' *IEEE Trans. on Very Large Scale Integr. (VLSI) Syst.*, 2008, **16**, 9, pp. 1127–1140
- 6 Khan, O. *et al.*: 'DCC: a dependable cache coherence multicore architecture,' *Comput. Archit. Lett.*, 2011, **10**, 1, pp. 12–15
- 7 Powell, M.D. *et al.*: 'Architectural core salvaging in a multi-core processor for hard-error tolerance,' *ACM SIGARCH News*, 2009, **37**, 3, pp. 93–104
- 8 Eggers, S. *et al.*: 'Simultaneous multithreading: a platform for next-generation processors,' *IEEE Micro*, 1997, **17**, 5, pp. 12–19
- 9 Hwu, W.W., and Patt, Y., 'Checkpoint repair for high-performance out-of-order execution,' *IEEE Trans. Comput.*, 1987, **C-36**, 12, pp. 1496–1514
- 10 Muralimanohar, N. *et al.*: 'Optimizing NUCA organizations and wiring alternatives for large caches with CACTI 6.0,' *Proc. of 40th IEEE/ACM Symp. on Microarchitecture*, Chicago, IL, USA, December 2007, pp. 3–14
- 11 Sawalha, L. *et al.*: 'Phase-guided scheduling on single-ISA heterogeneous multicore processors,' *Proc. of 14th Euromicro Conf. on Dig. Sys. Design*, Oulu, Finland, August–September 2011, pp. 736–745